ARCHITECT'S REFERENCE · ASE AGENTIC SYSTEM

ASE System Reference

What is implemented today, how a run flows end to end, where each mechanism lives in the code, and what the planned increments add. Written to be re-read before each work session.

State as of 28 October 2026, main at `1705f05`, increment I0 delivered (2026-09-21) plus the projects workspace (2026-10-15). Paths are relative to `ASE Agentic System/` unless noted.

Status at a glance

I0
increment delivered; I1 to I6 exist as models only

8 steps
in the generated I0 orchestrator

34
registered perspective checks (docs say 36)

6
RAG patterns in the library; 13 stage kinds

2 graphs
`ase` (control) and `ase-designs`

6 xfails
in RS-01, each naming the increment that clears it

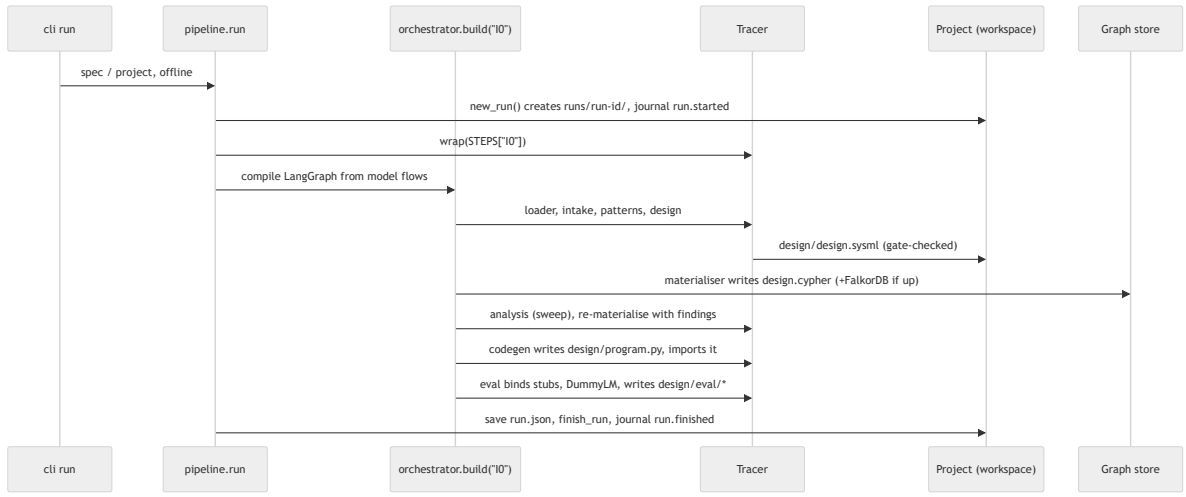
Each IDEF0 activity and ASE extension, and what currently performs it:

ACTIVITY	STEP	TODAY	IMPLEMENTATION	REPLACED IN
A1 Consolidate inputs	loader	`real`	`spec.read_spec` : structural reader of RS markdown, derives budgets from sentences	I1 extends
A1	intake	`stub`	ScriptedAnswers: applies the oracle's §2.1 answers, asks nothing	I1 IntakeAgent
A2 Scenarios	(none)	`absent`	Not in the I0 topology	I2 ScenarioAgent
A3 Styles / patterns	patterns	`stub`	RuleSelector: keyword table in `design.py` , defaults to NaiveRag	I2 PatternAgent

ACTIVITY	STEP	TODAY	IMPLEMENTATION	REPLACED IN
A4 Candidate architecture	design	stub	PatternCopier: copies the pattern into `design.sysml`, budgets overwritten, gate-checked	I2 DesignAgent
MECH	materialiser	real	`materialise.py`: the only writer of the design graph	I1 / I4 extend
A5 Explore options	analysis	partial	SweepOnly: analysis + 34 checks, no option exploration	I3 AnalysisAgent
A6 / A8 Evaluate, revisit	(none)	absent	No human decision step yet	I3 EvidenceAgent
R1 Code	codegen	real	`codegen.generate`: LangGraph + DSPy program; retrieval stages bound from outside	I3 / I4 extend
R1 Evaluate	eval	stub	StubEvaluator: one metric on five placeholder questions, word-overlap retriever	I4 EvalAgent
R1 Optimise, deploy	(none)	absent	`optimise()` is generated but never invoked; no Deployer	I4

End-to-end run

`python -m sysmlagents.cli run --project RS-01 --offline` walks this path. The topology comes from `AseWorkbenchI0` in `models/ase-evolution.sysml`; there is no back edge at I0 (`maxIterations = 1`).



#	STEP	REALISES	READS	WRITES
01	loader	A1	spec markdown	record: spec id, title, requirements. No files
02	intake	A1	spec §2.1 answers	answers_applied (7 for RS-01), questions_asked = []
03	patterns	A3	spec text, `models/rag-patterns.sysml`	pattern, rationale, rejections, ranked
04	design	A4	library, derived budgets	`design/design.sysml` ; gate_ok, budgets
05	materialiser	MECH	spec, design model, selection	`design/design.cypher` ; FalkorDB `ase-designs` when reachable
06	analysis	A5	design model	analysis numbers, verdicts, findings; then re-materialises with Finding nodes (`materialiser(findings)`)
07	codegen	R1	design model	`design/program.py` , imported as a module
08	eval	R1	program, spec, `inputs/corpus` if any	`design/eval/eval-report.json|md` , `questions.json` ; metrics

Reference result for RS-01 (offline): NaiveRag, 1 LLM call, 1,520 ms, 2,200 tokens, bounded and provenance-preserving, all four verdicts satisfied, sweep 8 warnings and 5 info, design graph 43 nodes and 51 edges, groundedness 0.40 traced to FR-2 on placeholder text.

Variants

- `--stop-after STEP` runs through `trace.Session.run_until` instead of the compiled LangGraph; same order, one step at a time.
- `--out DIR` runs outside projects (used by L2 tests); the graph partition key becomes the spec id.
- With neither `--offline` nor `--online`, the run is offline when `OPENAI_API_KEY` is unset.

Module map

All under `sysmlagents/`. `_paths.py` puts `AI Skills` on `sys.path` and resolves data files relative to the project, so code works from any working directory.

LAYER	MODULE	RESPONSIBILITY
Package	`__init__.py`	Re-exports; `validator()` runs the skill gate `check_model.py` through the SkillPort, no LLM
Intake	`spec.py`	`Spec`, `read_spec`: sections, FR/QR/C requirements, stakeholders, data, scripted answers, derived budgets
Design	`design.py`	`rule_select` (keyword table, `RULES`, `PATTERN_CARDS`), `copy_pattern` writes the design package
Graph writer	`materialise.py`	`build_partition` (design LPG), `write` (script + optional FalkorDB load), `replace_partition`
Reading models	`model_reader.py`	Subset extractor: definitions, requirements, patterns (usages, flows, satisfy, unallocated). Not a parser; the gate decides well-formedness
Analysis	`analysis.py`	`analyse_pattern`: calls, latency (critical path), tokens, bounded, provenance, verdicts; `compare`, `conformance`, `viable`
Checks	`kg.py`, `checks.py`	In-memory check graph and registry; 34 `@registry.check` functions
Method	`methodology.py`, `workflow.py`	Read the viewpoint/perspective catalogue; `walk()` runs the methodology workflow and sweep
Proof	`selfmodel.py`, `selftest.py`	L1 verify (`cli selftest`); a separate "every check can fire" test over the tripwire model
Teaching	`explain.py`, `diagram.py`	Derivation tables, SVG and Mermaid diagrams of a pattern

LAYER	MODULE	RESPONSIBILITY
Codegen	`codegen.py`	`generate` , `generate_all` ; `STAGE_CONTRACTS` (7 LLM kinds), `RETRIEVAL_STUBS` (6 kinds)
Runtime	`evaluate.py` , `runtime_stub.py`	Stub evaluator; `StubCorpus` , stub stage implementations, `offline_lm` (DummyLM), `configure_lm`
Orchestration	`pipeline.py` , `orchestrator.py`	`RunRecord` , step functions, `STEPS` ; topology and LangGraph built from the models; `realises()`
Inspection	`trace.py` , `project.py`	Per-step trace folders, `Session` ; workspace and projects
Graph layer	`semantic.py` , `lpg.py` , `falkor.py` , `graphstore.py`	SysML to semantic graph to property graph; FalkorDB load and `AseGraphRAG` ; one API over memory and FalkorDB, `parity()`
Rendering	`inspector.py`	All views shared by the Streamlit app and the notebooks. New views go here
Interfaces	`cli.py` , `mcp_server.py` , `launcher.py`	CLI; `ase` MCP server (11 typed tools); `up` / `down` for Docker, FalkorDB, graphs, app
Proof of concept	`agent.py`	`SysMLAgent` : LangChain ReAct agent writing SysML via skills. Not used by the pipeline

Orchestrator from models

The running workbench is generated, not hand-wired. This is what "ASE is modelled in ASE" means in code.

1. `orchestrator.self_model()` reads `ase-system.sysml` and `ase-evolution.sysml` together.
2. `topology(increment)` takes the `AseWorkbench<inc>` part usages as nodes, its flows as edges, finds back edges by DFS, and reads `maxIterations` .
3. `build(increment, steps)` maps each usage to a registered step function (a missing one raises `KeyError`), adds forward edges, and turns each back-edge tail into a conditional edge that retries while `rec.rework` is set and passes < `maxIterations` .
4. `realises(increment)` reads `@Realises` metadata so every trace folder is labelled with its IDEF0 activity.

What `cli selftest` asserts

#	ASSERTION
1	Gate: `ase-system.sysml`, `ase-evolution.sysml` and both together pass the structural gate
2	Aggregates: each agent's declared calls, latency and tokens equal the analysis of its pipeline (HierarchyGap, checked by hand until I3)
3	Sweep: no error-severity finding on the self-model
4	Orchestrator: for each built increment (only I0), the compiled graph's edges equal the model's topology
5	Realises: every part of every `AseWorkbenchI*` has exactly one activity in A1 to A8, MECH or R1

Adding a step means: add the part and flow to `ase-evolution.sysml`, add its `@Realises` entry, register the function in `pipeline.STEPS`, and selftest must still pass.

Projects, runs, traces

Every design is a project in the workspace (`ASE_WORKSPACE`, default `workspace/`, gitignored). `examples/RS-01/` is the committed reference project that tests read.

```
workspace/<project-id>/
  project.json        id, spec, increment, status, last_run, timestamps
  spec/               the spec copied in
  inputs/             corpus (copied from <spec dir>/data/<RS>/corpus if present)
  design/             design.sysml, design.cypher, program.py, eval/, run.json (la
  runs/<YYYYmmdd-HHMMSS>/
    run.json  run.log
    trace/index.json
    trace/NN—<ACT>-<step>/  step.json, model.sysml, diagram.svg|.mmd,
                            graph.before.json, graph.after.json, graph.delta.cyphe
  journal.jsonl       project.created, run.started, run.finished|failed, notes.upd
  notes.md
```

RunRecord (`pipeline.py`) carries run and project ids, intake results, selection, design path and gate verdict, analysis and verdicts, findings, graph counts, program path, eval metrics, `steps_run` and `errors`. Underscored fields (`_spec`, `_design_model`, `_module`, …) are runtime-only and not serialised.

step.json records the ICOM frame of the activity, record changes, findings, decisions added, graph before/after counts, changed artifacts, and the LLM calls taken from `dspy.settings.lm.history`.

Knowledge graph

GRAPH	BUILT BY	LABELS	EDGES
`ase` (control)	`lpg.project_lpg()` over `methodology/*.sysml`	Activity, Viewpoint, Perspective, Concern, Stakeholder, Item, Check, Agent (+ Sourced, Extension, ReviewOnly)	CONSUMES, CONTROLLED_BY, PRODUCES, PERFORMED_BY, ASSIGNED_TO, RETRIEVES, DECOMPOSES_TO, DETAILS, DECLARES, FRAMES, EXPOSES, HELD_BY, SATISFIES, CONTAINS, FLOWS_TO, PRECEDES, SPECIALIZES
`ase-designs`	`materialise.build_partition`, keyed by project id	Spec, Stakeholder, SpecRequirement, Decision, Design, Pattern (shared, merged), Stage, Requirement, Finding, Run	HAS_STAKEHOLDER, STATES, DECIDED, SUBMITTED_AS, INSTANTIATES, CONTAINS, FLOWS_TO, SATISFIES, UNALLOCATED, TRACES_TO, CHOSE, FOUND, PRODUCED, READ

- **MemoryStore** supports a read-only Cypher subset: one MATCH with one linear path, WHERE with comparison and string operators, RETURN [DISTINCT], `count`, `labels`, `type`, `toLower`, `toUpper`, `size`, `coalesce`, ORDER BY a returned column, LIMIT. Writes, CALL, UNWIND, OPTIONAL, variable-length paths and comma patterns raise `UnsupportedCypher`.
- **FalkorStore** connects via `FALKOR_HOST` / `FALKOR_PORT` (localhost:6379). Docker image `falkordb/falkordb:v4.20.1`, container `ase-falkordb`, Browser on port 3000.
- **parity()** compares counts and 10 fixed queries across both backends. `cli up` loads `ase`, rebuilds `ase-designs` from every project's `design.cypher`, then checks parity.
- `ASE_NO_FALKOR=1` forces the script-only path; tests set it.

Analysis and checks

`analysis.analyse_pattern` computes a pattern's properties from declared stage estimates:

- **Cycles**: back edges found by DFS; cycle members are multiplied by `maxIterations`.

- **Fan-out**: multiplies the cost of everything downstream.
- **Latency**: critical path over the DAG of forward edges.
- **Verdicts**: LatencyBudget, CostBudget, GroundedAnswer (provenance on every retrieval-to-answer path), BoundedLoop. Labels: satisfied, CLAIMED BUT FAILS, holds unclaimed, correctly unallocated.

`kg.project()` builds a separate in-memory check graph joining design and methodology; `checks.py` registers 34 checks across 13 perspectives; `workflow.walk()` runs them in the methodology's order (Context, Functional, Information, Knowledge, Concurrency, Autonomy, Deployment, Operational, Development, then the perspective sweep). A check that raises becomes a warning finding rather than crashing the run.

`python -m sysmlagents.selftest` (not the CLI command) proves every check can fire, using `models/tripwire.sysml`, a deliberately defective design.

Code generation

`codegen.generate(model, pattern)` emits one self-contained `program.py`. Same design and generator version give the same bytes.

PART OF PROGRAM.PY	CONTENT
Header	"Generated from … do not edit by hand", analysis summary, verdicts
Constants	`MAX_ITERATIONS`, `BUDGET_MS`, `BUDGET_TOKENS` from the design's requirements; `IMPLEMENTATIONS` registry (empty, bound by stage kind from outside)
State	`RagState`: question, queries, chunks (each with `source_id`), answer, citations, deficit, iteration, spent_tokens
Signatures	One `dspy.Signature` per LLM stage from `STAGE_CONTRACTS`; a kind with no contract gets a marked TODO signature
Program	`<Design>Program(dspy.Module)` with a `ChainOfThought` per LLM stage; `forward()` runs the graph
Non-model stages	Functions that call `IMPLEMENTATIONS[kind]` or raise `NotImplementedError`: DenseRetriever, SparseRetriever, GraphRetriever, ReciprocalRankFusion, CrossEncoderReranker, WebSearchFallback
Graph	`build_graph()`: LangGraph `StateGraph`; `hasFallback` with two successors becomes a conditional edge; back edges become bounded retry branches

PART OF PROGRAM.PY	CONTENT
Optimisation	`metric()` for GEPA feedback; `optimise()` calls `dspy.GEPA` with the reflection model; `configure()` sets `dspy.LM`

`generate_all()` writes the six library programs to `generated/`. Programs for RerankHybridRag and AgenticRag currently fail at run time with a LangGraph concurrent-update error on their parallel branches.

Evaluation today

- **Questions**: five placeholders, one per functional requirement plus an out-of-corpus question ("What is the weather like on the moon?").
- **Corpus**: `<project>/inputs/corpus/*.txt` split on blank lines; otherwise the spec's own sentences, labelled as placeholder.
- **Retrieval**: word overlap, top 3, bound into `IMPLEMENTATIONS` by `runtime_stub.bind`.
- **Model**: offline DummyLM with a canned answer citing the first chunk; online `dspy.LM(SYSMLAGENTS_MODEL)`.
- **Metric**: one, groundedness (citations non-empty and all among retrieved ids), traced to the first grounding requirement.
- Outputs: `design/eval/eval-report.json`, `eval-report.md`, `questions.json`. No optimisation is run.

CLI, MCP, app, notebooks

CLI (`python -m sysmlagents.cli`)

COMMAND	DOES
`project new <spec> [--id --name --description --owner]`	Create a project from a spec
`project list` · `project show <id>`	List projects; show meta, runs, last journal entries, files
`run [spec] [--project] [--offline|--online] [--out] [--stop-after] [--no-trace]`	Run the pipeline; exit 1 on step errors

COMMAND	DOES
`trace <project> [--run]`	Per-step activity, status, time, graph delta, files, changed fields
`status [project]` · `explain <project>`	Latest run summary; derivation of the design's numbers
`selftest`	L1 self-model verification
`up [--no-app] [--port]` · `down`	Docker, FalkorDB, graphs, parity, Streamlit; stop FalkorDB

MCP server `ase` (11 tools)

`ase_step` (full ICOM frame), `ase_controls`, `ase_process`, `ase_viewpoint`, `ase_perspective`, `ase_concern`, `ase_obligations`, `ase_gap` (required attributes a design lacks), `ase_provenance`, `ase_status`, `ase_reload`. Typed tools only, no Cypher surface; falls back to the in-memory projection without FalkorDB.

Streamlit workbench (`app/`)

PAGE	SHOWS
Home	Docker, FalkorDB, built increments, graph counts and parity, model settings, projects
0 Projects	Create, browse; tabs for runs, files, journal, notes, settings
1 Process	IDEF0 diagram coloured real/stub/absent, ICOM frame per activity
2 Run	Step a run: next step, run until, run all; scripted stakeholder panel
3 Step Inspector	Per step: ICOM, SysML, code and files, graph delta, findings, record, LLM calls, tests
4 Knowledge Graph	Partition and backend choice, counts, parity, Cypher console
5 Evolution	I0 to I6 table, per-increment numbers, deltas, xfail burn-down
6 Specs	RS-01 to RS-10, submitted part, oracle behind a toggle
7 Methodology	Viewpoints, perspectives, concerns, activities and agents, checks

Notebooks

`notebooks/I0/` is generated by `notebooks/build_i0.py` (edit the builder, never the notebooks): 00 start here, 01 self-model and evolution, 02 run end to end, one notebook per activity (A1, A3, A4, M, A5, R1), and 99 oracle RS-01. Each activity notebook follows the same template: ICOM, run, code, record before/after, SysML, graph delta, findings, oracle, component history.

Tests

`tests/conftest.py` sets `ASE_NO_FALKOR=1` and a temporary `ASE_WORKSPACE`. No test calls a model. No L3 (`slow`) test exists yet.

FILE	LAYER	COVERS
`test_selfmodel.py`	L1	`verify()` returns no issues
`specs/test_rs_01.py`	L2	RS-01 oracle: no forbidden questions, NaiveRag chosen with reasons, gate and budgets, `models/` hashes unchanged, numbers 1/1520/2200, graph content and trace edges, program shape, eval report
`test_trace.py`	L1/L2	Activity labels, folder order, ICOM frames, graph deltas, artifacts, Session equals LangGraph
`test_project.py`	unit	Workspace layout, ids, runs kept, partition per project, examples
`test_graphstore.py`	unit	Cypher subset, unsupported queries, script round trip; FalkorDB parity (skipped without it)
`test_app.py`	UI	Every page renders; Run page steps a project end to end
`test_notebooks.py`	UI	Executes every notebook offline (about 1.5 min)
`test_pattern_catalogue.py`	unit	55 patterns valid; catalogue check names equal the registry

Strict xfails in RS-01 (the burn-down list)

TEST	CLEARED BY
`test_expected_questions_asked`	I1 elicitation

TEST	CLEARED BY
`test_selection_cites_graph_query`	I2 selection by graph query, recorded
`test_spec_requirements_in_design`	I2 spec requirements as requirement defs in the design
`test_abstention_path`	I3 abstention as a distinct outcome
`test_real_retriever`	I4 real dense retriever over the corpus
`test_full_metric_set`	I4 five metrics, 100+ synthetic examples, GEPA before/after

Configuration

VARIABLE	DEFAULT	EFFECT
`SYSMLAGENTS_MODEL`	`openai/gpt-5-mini`	Model for generated programs and online eval (`provider/model`)
`SYSMLAGENTS_REFLECTION_MODEL`	`openai/gpt-5`	GEPA reflection model
`OPENAI_API_KEY`	unset	Its absence makes runs offline by default
`ASE_WORKSPACE`	`workspace/`	Projects root
`ASE_NO_FALKOR`	unset	`1` skips FalkorDB loads
`FALKOR_HOST` / `FALKOR_PORT`	localhost / 6379	FalkorDB connection
`ASE_DOCKER_DESKTOP`	Docker Desktop default path	Used by `cli up` to start Docker
`PYTHONIOENCODING`	set to utf-8 by launchers	Needed on Windows

Ports: Streamlit 8501, FalkorDB 6379, FalkorDB Browser 3000. Python env: conda `DATAENLIGHT_AI_ARCH_PATTERNS`.

Known limits and drift

Code issues worth fixing

- **Provider coupling.** `configure_lm` decides offline/online on `OPENAI_API_KEY` only; `agent.py` imports `langchain_openai` directly; `requirements.txt` names an `llm.py` that doesn't exist. Plan item 0.8 (provider-agnostic model layer) is not started.
- **Parallel branches.** RerankHybridRag and AgenticRag programs crash with a LangGraph concurrent-update error. A fan-in fix exists in PR #1.
- **New state keys are dropped.** `RagState` has no `verdict` key, so LangGraph discards it: in CorrectiveRag the web fallback always runs. Any new stage output (entities, subgraph) must be added to the state schema.
- **Loop bound not enforced.** `_branch` increments `iteration` inside a routing function, which LangGraph does not persist; an always-ungrounded SelfRag hit `GraphRecursionError` after 3,334 calls instead of stopping at 3.
- **Token budget stop never fires.** Generated programs initialise `spent_tokens` but never increment it.
- `run_graph` recompiles the graph for every question.
- The Run node id uses the start timestamp, not the run id (`pipeline.py:205`).
- `spec._derive_budgets` checks a key that is never set in the minutes branch (`spec.py:156`).
- The offline DummyLM cites the first chunk for every question, so groundedness 0.40 measures retrieval overlap, not answer quality.
- The materialiser runs twice per run; the second pass is untraced as its own step.

Documentation drift

- 36 checks in `ase-system.sysml` , `README.md` and the roadmap; 34 in code.
- The MCP server docstring says eight tools; there are 11. The selfmodel docstring says five assertions, then four.
- The I0 `@Delta` still names `designs/<spec>/design.sysml` ; runs now write to `<project>/design/` .
- `examples/RS-01/` predates projects: its `run.json` has no project or run id.
- IDEF0 agent names (ConsolidationAgent, ViewAuthoringAgent) differ from the system model (IntakeAgent, DesignAgent).
- The Deployer is in the roadmap but not yet in either model.
- `docs/ase-architecture.html` was generated on 21 September 2026, before the evolution model last changed (12 October).

Planned phases

The roadmap's increments are the destination. Each starts by updating `ase-evolution.sysml` (`@Delta` , `@Realises`) before code, and ends with the review checklist.

INC	ADDS	DEFINITION OF DONE
I1	IntakeAgent; `RequirementsSpec` model; deterministic gap checklist from `PerformanceOnFunctional.requiredAttributes` and viewpoint concerns; LLM only words questions; `Stakeholder` interface (Console, Scripted); assumptions as Decision nodes	RS-01 §2.1 exact (asked ⊆ questions, forbidden ∩ questions = ∅); RS-02 version question asked; I0 still passes
I2	ScenarioAgent, PatternAgent, DesignAgent; `@PatternForm` projected to Pattern nodes; selection = Cypher query + rationale from retrieved fields; instantiation `part def <Spec>Design :> <Pattern>`	RS-02 rationale cites fields and query; RS-03 waiver recorded
I3	AnalysisAgent with `explore()` options table; EvidenceAgent; rework loop (max 3); minimal composition; three-way conditional codegen; security checks on trust boundaries	RS-06 options table and no design before decision; RS-04 conditional edge; RS-05 bounded loop
I4	`rag_runtime/` (dense, BM25, RRF, rerank, filters); recipe-driven eval datasets with splits; GEPA with held-out gate; deterministic Deployer and deploy gate; `@Measured` feedback	RS-05 thresholds; RS-01 to 04 L3; RS-01 deploys in one command; degraded candidate refused; byte-identical regeneration
I5	Composer (bounded search, ≤3 candidates); workloadClass; trust and tenancy; graph retriever with parameterised Cypher; runtime patterns in deployment templates; promotion workflow	RS-07, RS-08 pass with `models/` unchanged; RS-03b after promotion
I6	Agent-as-part vocabulary; evidence packs; release gate; reproducibility harness; ASE's own prompts optimised; multi-agent deploy, canary, rollback	RS-09, RS-10 pass; all ten specs pass; ASE evidence pack produced by the pipeline

Near-term reordering under discussion. The incremental delivery plan pulls a thin slice of I4 forward (real retrieval, eval, local Docker deploy) so pilot clients get working agents sooner. The 4-week GraphRAG development plan does the same for a naive GraphRAG pattern on FalkorDB. Both keep the non-negotiables and land each slice in the evolution model first.

Command sheet

```
# from ASE Agentic System/
python -m sysmlagents.cli project new docs/specs/RS-01-hr-policy-assistant.md [--i
python -m sysmlagents.cli run --project RS-01 --offline [--stop-after design]
python -m sysmlagents.cli trace RS-01 [--run ID]
python -m sysmlagents.cli status | explain RS-01 | selftest
python -m sysmlagents.cli up [--no-app]      # Docker + FalkorDB + graphs + app
python -m sysmlagents.cli down
python -m sysmlagents.selftest               # every check can fire (tripwire)
python -m sysmlagents.mcp_server             # ase MCP server
streamlit run app/Home.py                    # app alone, in-memory graphs
python notebooks/build_i0.py                 # regenerate notebooks
python docs/build_architecture_page.py       # regenerate docs/ase-architecture.ht

python -m pytest tests -q -c tests/pytest.ini
python -m pytest tests -m "not slow" -c tests/pytest.ini
python -m pytest tests/test_notebooks.py -c tests/pytest.ini

# whole repo, from the repo root
cd "AI Skills" && python -m ontoagents verify          # expect VERIFY OK
cd "ASE School/build" && python build_reference.py && python build_scripts.py
```